

S2.01 – Développement d'une application

14/06/2026

Le jeu FindMyWord

Abasse KADERBHAJ

BUT Informatique 1re année
Groupe Cérés

Table des matières

1. Introduction.....	2
2. Objectif du projet.....	2
3. Lancement du programme	2
4. Organisation des fichiers	3
5. Modélisation UML	4
Diagramme UML simplifié	4
Diagramme UML détaillé.....	4
6. Description des classes.....	6
Classe <i>Main</i>	6
Classe <i>Game</i>	6
Classe <i>Word</i>	6
Classe Tentative	6
Classe Affichage	7
Classe <i>Joueur</i>	7
Classe <i>Saisie</i>	7
Interface WordRepository	7
Classe WordRepositoryAleatoire	8
Classe WordRepositoryFixe	8
7. Choix réalisés.....	8
Utilisation d'une interface pour les mots	8
Ajout d'un clavier affiché.....	8
Ajout des couleurs.....	8
Ajout d'une option pour recommencer	9
Ajout de tests unitaires simples	9
8. Fonctionnement du jeu	9
9. Tests réalisés.....	10
10. Bilan	11
Ce qui a été fait.....	11
Ce qui fonctionne	11
Ce qui n'a pas été fait	11
Ce qui ne fonctionne pas ou les limites connues	11
11. Difficultés rencontrées	12
12. Organisation du travail.....	12
13. Conclusion	12

1. Introduction

Dans le cadre de cette SAÉ, nous avons développé un jeu en Java appelé FindMyWord. Le principe est proche du jeu Wordle : le joueur doit retrouver un mot secret de cinq lettres en six tentatives maximums.

Après chaque tentative, le programme analyse le mot proposé et indique, pour chaque lettre, si elle est bien placée, présente mais mal placée, ou absente du mot secret. Le but était donc de créer un programme fonctionnel, mais aussi de structurer le code correctement avec les principes de la programmation orientée objet.

Ce projet nous a permis de travailler plusieurs points importants : la création de classes, la séparation des responsabilités, l'utilisation d'une interface, la gestion d'un fichier de mots, l'affichage en console et la mise en place de tests simples.

2. Objectif du projet

L'objectif principal était de développer une version simplifiée du jeu Wordle.

Le programme devait permettre de :

- Demander le nom du joueur ;
- Choisir un mot secret de cinq lettres ;
- Laisser le joueur proposer un mot ;
- Vérifier que la tentative est valide ;
- Analyser chaque lettre avec les résultats OK, PRESENT ou ABSENT ;
- Afficher la grille de jeu ;
- Gérer les six tentatives maximums ;
- Détecter la victoire ou la défaite ;
- Afficher un message de fin de partie.

Une tentative invalide ne doit pas être comptabilisée comme un essai. Par exemple, si le joueur entre un mot trop court, trop long, avec un chiffre ou avec des lettres répétées, le programme affiche une erreur et redemande un mot.

3. Lancement du programme

Le programme peut être lancé depuis un terminal, sans ouvrir d'environnement de développement comme Eclipse ou IntelliJ.

Depuis le dossier principal du projet, il faut d'abord compiler le programme.

Sur Windows PowerShell:

```
javac -encoding UTF-8 -cp "jar/wordset.jar" -d out src\findmyword\*.java  
java -cp "out;jar/wordset.jar" findmyword.Main
```

Sur Linux ou Mac:

```
javac -encoding UTF-8 -cp jar/wordset.jar -d out src/findmyword/*.java  
java -cp out:jar/wordset.jar findmyword.Main
```

Pour lancer les tests unitaires sur Windows PowerShell:

```
javac -encoding UTF-8 -cp "jar/wordset.jar" -d out src\findmyword\*.java  
tests\findmyword\TestUnitaire.java  
java -cp "out;jar/wordset.jar" findmyword.TestUnitaire
```

Le fichier README.md reprend également ces commandes afin de faciliter le lancement du projet.

4. Organisation des fichiers

Le projet est organisé de manière à séparer clairement le code source, les tests, les données et la librairie utilisée pour lire les mots.

```
FindMyWord/
├── data/
│   └── mots.json
├── jar/
│   └── wordset.jar
├── src/
│   └── findmyword/
│       ├── Affichage.java
│       ├── Game.java
│       ├── Joueur.java
│       ├── Main.java
│       ├── Saisie.java
│       ├── Tentative.java
│       ├── Word.java
│       ├── WordRepository.java
│       ├── WordRepositoryAleatoire.java
│       └── WordRepositoryFixe.java
├── tests/
│   └── findmyword/
│       └── TestUnitaire.java
└── README.md
```

Le dossier src contient le code principal du projet. Les classes sont placées dans le package findmyword afin de mieux organiser le programme.

Le dossier tests contient la classe TestUnitaire, utilisée pour vérifier les méthodes importantes du projet.

Le dossier data contient le fichier JSON avec la liste des mots utilisés par le jeu.

Le dossier jar contient la librairie wordset.jar, fournie pour lire le fichier de mots.

Le fichier README.md explique rapidement le rôle du projet, son organisation et les commandes pour compiler, lancer le jeu et exécuter les tests.

5. Modélisation UML

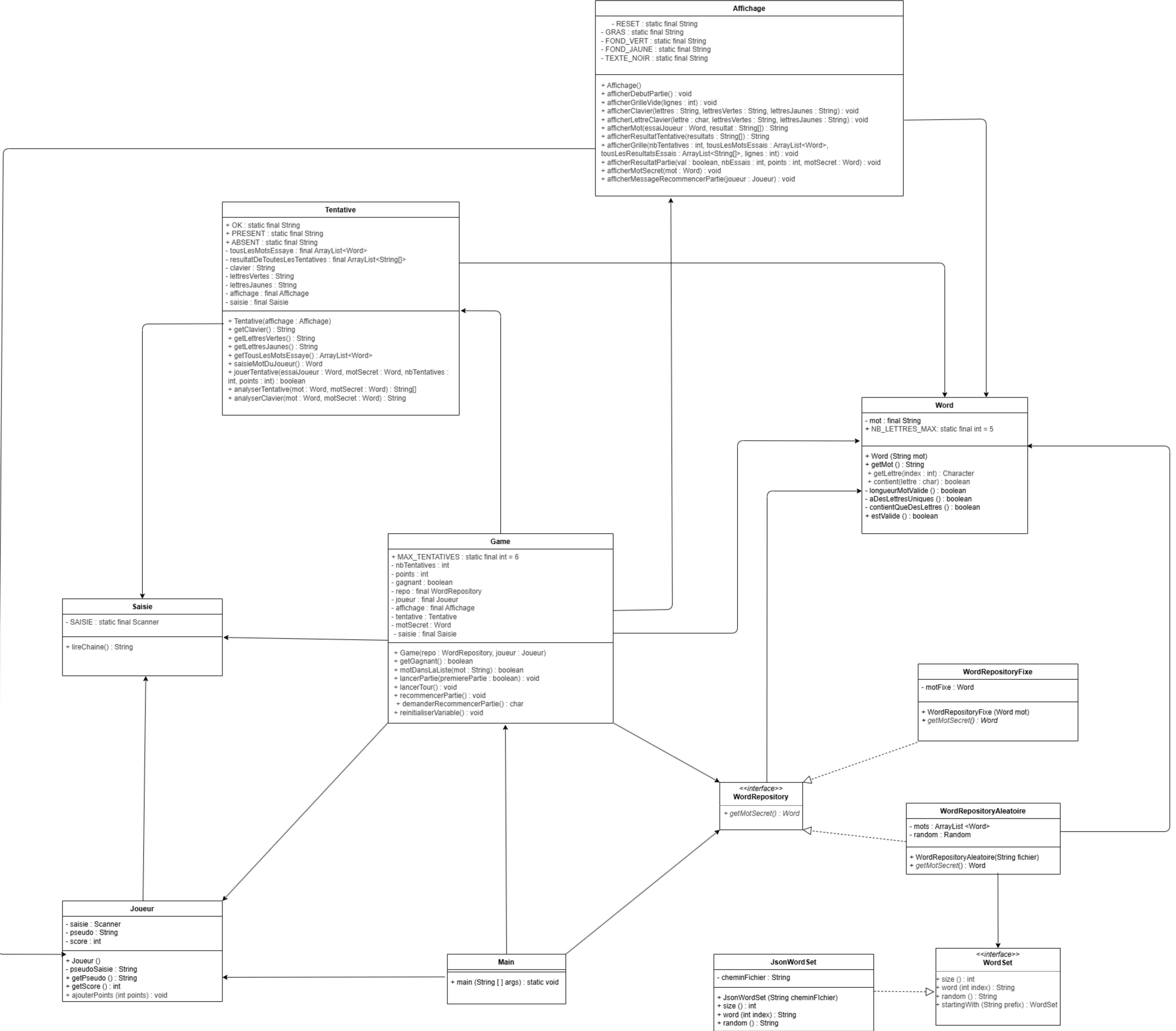
Diagramme UML simplifié

Ce premier diagramme présente uniquement les liens principaux entre les classes. Il permet de voir l'organisation générale du projet.

```
graph TD
    Main --> WordRepositoryAleatoire
    Main --> Joueur
    Main --> Game
    Game --> WordRepository
    Game --> Joueur
    Game --> Affichage
    Game --> Tentative
    Game --> Word
    Game --> Saisie
    Tentative --> Affichage
    Tentative --> Saisie
    Tentative --> Word
    Affichage --> Joueur
    Affichage --> Saisie
    Affichage --> Word
    Joueur --> Saisie
    WordRepository --> Word
    WordRepositoryAleatoire --> WordRepository
    WordRepositoryFixe --> WordRepository
    WordRepositoryAleatoire --> WordSet
    JsonWordSet --> WordSet
```

Diagramme UML détaillé

Le diagramme détaillé ajoute les attributs et les méthodes principales des classes.



6. Description des classes

Classe *Main*

La classe *Main* est le point d'entrée du programme. Elle crée les objets nécessaires au lancement du jeu.

Elle crée d'abord un *WordRepositoryAleatoire*, qui permet de récupérer un mot secret depuis le fichier JSON. La classe *WordRepositoryFixe* existe aussi dans le projet, mais elle sert surtout aux tests. Ensuite, elle crée un joueur, puis une partie avec la classe *Game*.

Cette classe contient aussi un bloc *try/catch*, car la lecture du fichier de mots peut provoquer une erreur si le fichier n'est pas trouvé ou si le fichier ne peut pas être lu correctement.

Classe *Game*

La classe *Game* gère le déroulement général de la partie.

Elle contient notamment :

- Le mot secret ;
- Le nombre de tentatives ;
- Les points ;
- L'état de victoire ;
- Le joueur ;
- La tentative en cours ;
- L'affichage.

Elle s'occupe de lancer la partie, d'enchaîner les tours, de vérifier si le joueur a gagné ou perdu, puis de proposer une nouvelle partie à la fin.

Classe *Word*

La classe *Word* représente un mot du jeu.

Elle permet de vérifier les règles de base d'un mot :

- Le mot doit contenir exactement cinq lettres ;
- Il doit contenir uniquement des lettres ;
- Il ne doit pas contenir de lettres répétées.

Classe *Tentative*

La classe *Tentative* gère les essais proposés par le joueur.

Elle permet de :

- Saisir un mot ;
- Vérifier si le mot est valide ;
- Analyser le mot par rapport au mot secret ;
- Enregistrer les tentatives déjà jouées ;
- Enregistrer les résultats de chaque tentative ;
- Mettre à jour le clavier affiché.

L'analyse d'une tentative renvoie un tableau de résultats. Pour chaque lettre, le programme indique :

- OK si la lettre est à la bonne position ;
- PRESENT si la lettre est dans le mot mais pas à la bonne position ;
- ABSENT si la lettre n'est pas dans le mot secret.

Classe Affichage

La classe *Affichage* s'occupe de tout ce qui est affiché dans le terminal.

Elle affiche :

- Le message de bienvenue ;
- La grille ;
- Les mots proposés ;
- Les résultats ;
- Le clavier ;
- Les messages de victoire ou de défaite ;

Cette classe permet de séparer l'affichage de la logique du jeu. Par exemple, *Game* décide ce qui doit se passer, mais *Affichage* décide comment le montrer à l'utilisateur.

Classe Joueur

La classe *Joueur* représente le joueur.

Elle stocke :

- Le pseudo du joueur ;
- Son score.

Le pseudo est demandé au début de la partie. Le score est mis à jour lorsque le joueur trouve le mot secret.

Classe Saisie

La classe *Saisie* permet de gérer les entrées clavier.

Elle utilise un *Scanner* pour lire les réponses de l'utilisateur. Nous avons créé cette classe pour éviter de répéter directement le *Scanner* dans plusieurs classes différentes.

Cela rend aussi le code plus simple à lire, car la saisie est isolée dans une classe à part.

Interface WordRepository

WordRepository est une interface qui définit les méthodes nécessaires pour récupérer un mot secret et vérifier si un mot appartient à la liste.

Elle permet d'utiliser plusieurs manières de récupérer un mot sans modifier la logique du jeu.

C'est ici que le polymorphisme est utilisé. La classe *Game* dépend de l'interface *WordRepository*, et non directement d'une classe précise. Cela permet d'utiliser soit un dépôt aléatoire pour le vrai jeu, soit un dépôt fixe pour les tests.

Classe WordRepositoryAleatoire

Cette classe permet de récupérer un mot secret aléatoire depuis le fichier JSON.

Elle est utilisée dans le vrai lancement du jeu. Elle s'appuie sur la librairie fournie dans le fichier *wordset.jar* pour lire les mots.

Classe WordRepositoryFixe

Cette classe retourne toujours le même mot secret.

Elle n'est pas utilisée pour la partie normale. Elle sert surtout pour tester le programme plus facilement.

Par exemple, si le mot secret est toujours *ligne*, il devient possible de vérifier précisément si les résultats OK, PRESENT et ABSENT sont corrects.

7. Choix réalisés

Utilisation d'une interface pour les mots

Nous avons choisi d'utiliser une interface *WordRepository* pour séparer la classe *Game* de la manière dont le mot secret est récupéré.

Cela rend le code plus souple. Pour jouer normalement, on utilise *WordRepositoryAleatoire*. Pour tester, on peut utiliser *WordRepositoryFixe*.

Ce choix permet aussi de montrer l'utilisation du polymorphisme, car *Game* peut fonctionner avec plusieurs classes différentes tant qu'elles respectent l'interface *WordRepository*.

Ajout d'un clavier affiché

Nous avons ajouté un clavier dans l'affichage pour aider le joueur à visualiser les lettres encore disponibles.

Les lettres absentes sont retirées du clavier. Les lettres présentes dans le mot secret sont affichées en couleur.

Ajout des couleurs

Pour améliorer l'affichage du clavier, nous avons cherché comment mettre de la couleur dans un terminal. Sur le site officiel *Ecma International*, nous avons découvert que les codes ANSI permettent de changer l'affichage du texte. Nous avons donc utilisé un fond vert pour les lettres bien placées, un fond jaune pour les lettres présentes mais mal placées, et nous avons laissé disparaître les lettres absentes.

Les couleurs ont été ajoutées uniquement pour améliorer la lisibilité.

Dans la grille :

- Les lettres OK sont affichées en vert ;
- Les lettres PRESENT sont affichées en jaune ;
- Les lettres ABSENT restent sans couleur particulière.

Dans le clavier :

- Les lettres bien placées apparaissent en vert ;
- Les lettres présentes mais mal placées apparaissent en jaune ;
- Les lettres absentes disparaissent.

Ajout d'une option pour recommencer

À la fin d'une partie, le programme demande au joueur s'il veut recommencer.

Cette fonctionnalité permet de rendre le programme plus agréable à utiliser. Cela évite de devoir relancer la commande Java à chaque nouvelle partie.

Ajout de tests unitaires simples

Une classe *TestUnitaire* a été ajoutée pour vérifier les parties importantes du projet.

Les tests portent principalement sur :

- La validation des mots avec la classe *Word* ;
- L'analyse des tentatives ;
- L'affichage logique du clavier ;
- Le fonctionnement de *WordRepositoryFixe*.

Nous n'avons pas utilisé JUnit, car l'objectif était de garder des tests simples, lisibles et faciles à lancer depuis le terminal.

Les tests affichent les valeurs attendues et les valeurs obtenues, afin de comprendre facilement ce qui est vérifié.

8. Fonctionnement du jeu

Au lancement, le programme demande le pseudo du joueur. Ensuite, il choisit un mot secret de cinq lettres dans le fichier de mots.

Le joueur propose ensuite un mot. Le programme vérifie d'abord si le mot est valide.

Un mot est refusé si :

- Il ne contient pas exactement cinq lettres ;
- Il contient autre chose que des lettres ;
- Il contient une lettre répétée ;
- Il n'est pas dans la liste des mots autorisés.

Si le mot est invalide, un message d'erreur est affiché et la tentative n'est pas comptée.

Si le mot est valide, le programme l'analyse par rapport au mot secret. La grille est ensuite mise à jour avec les couleurs et les résultats.

La partie se termine dans deux cas :

- Le joueur trouve le mot secret ;
- Le joueur utilise ses six tentatives sans trouver le mot.

En cas de victoire, le programme affiche le nombre d'essais utilisés et les points gagnés. En cas de défaite, il affiche le mot secret.

9. Tests réalisés

Plusieurs tests ont été réalisés pour vérifier le bon fonctionnement du programme.

Tests sur la validation des mots

Nous avons vérifié plusieurs cas :

- Un mot trop court ;
- Un mot trop long ;
- Un mot avec un chiffre ;
- Un mot avec des lettres répétées ;
- Un mot correct.

Ces tests permettent de vérifier que la classe *Word* applique bien les règles du jeu.

Tests sur l'analyse des tentatives

Nous avons utilisé un mot secret fixe, par exemple *ligne*, pour vérifier les résultats attendus.

Exemples de tests :

Mot secret : ligne

Mot testé : ligne

Résultat attendu : OK OK OK OK OK

Mot secret : ligne

Mot testé : genil

Résultat attendu : PRESENT PRESENT PRESENT PRESENT PRESENT

Mot secret : ligne

Mot testé : table

Résultat attendu : ABSENT ABSENT ABSENT PRESENT OK

Ces tests permettent de vérifier que les lettres sont correctement analysées.

Tests sur le clavier

Le clavier a été testé pour vérifier que les lettres sont bien mises à jour après une tentative.

Par exemple, avec le mot secret *ligne* et la tentative *table* :

- *t*, *a* et *b* doivent disparaître ;
- *l* doit être indiqué comme présent mais mal placé ;
- *e* doit être indiqué comme bien placé.

Tests sur WordRepositoryFixe

WordRepositoryFixe a été testé pour vérifier qu'il retourne bien le mot choisi à l'avance.

Cette classe sert surtout à stabiliser les tests.

10. Bilan

Ce qui a été fait

Nous avons réalisé une version jouable de FindMyWord.

Le programme permet de :

- Lancer une partie depuis le terminal ;
- Saisir le pseudo du joueur ;
- Choisir un mot secret depuis un fichier JSON ;
- Vérifier les mots proposés ;
- Refuser les tentatives invalides sans les compter ;
- Analyser les lettres avec OK, PRESENT et ABSENT ;
- Afficher une grille de jeu ;
- Afficher un clavier mis à jour ;
- Gérer la victoire ;
- Gérer la défaite ;
- Calculer les points ;
- Proposer de recommencer une partie.

Ce qui fonctionne

Les règles principales du jeu fonctionnent.

Les mots invalides sont bien refusés. Les tentatives invalides ne sont pas comptabilisées. Les résultats des lettres sont cohérents avec le mot secret. La grille garde l'historique des tentatives précédentes. Le programme détecte correctement la victoire et la défaite.

Le lancement depuis le terminal fonctionne également, à condition que le fichier *mots.json* et le fichier *wordset.jar* soient bien présents dans les bons dossiers.

Ce qui n'a pas été fait

Certaines fonctionnalités optionnelles n'ont pas été réalisées.

Nous n'avons pas ajouté :

- La gestion du temps ;
- Le mode deux joueurs ;
- L'historique complet des parties ;
- Le choix du nombre de manches.

Ces éléments étaient indiqués comme optionnels. Nous avons donc préféré nous concentrer sur une version simple, stable et conforme aux règles principales du jeu.

Ce qui ne fonctionne pas ou les limites connues

À ce stade, aucun bug bloquant n'a été identifié dans la version finale.

Cependant, il existe quelques limites :

- Le programme doit être lancé depuis le bon dossier pour trouver *data/mots.json* ;
- Les tests ne simulent pas une partie complète avec saisie clavier, ils testent surtout les méthodes importantes séparément.

Ces limites ne bloquent pas le fonctionnement principal du jeu.

11. Difficultés rencontrées

Une première difficulté a été de bien organiser les classes. Au début, certaines responsabilités étaient mélangées. Nous avons donc séparé progressivement la logique du jeu, l’affichage, la gestion des mots et la saisie.

Une autre difficulté a été l’utilisation du fichier JSON avec le fichier *.jar*. Il fallait bien comprendre son fonctionnement et comment lancer le programme depuis le terminal.

Enfin, l’affichage du clavier a aussi demandé plusieurs ajustements. Au départ, les lettres présentes étaient seulement mises en majuscules, mais ce n’était pas assez visible. Nous avons donc ajouté des couleurs pour rendre l’information plus claire.

12. Organisation du travail

Le travail a été réparti autour des grandes parties du programme.

Une partie du travail a été consacrée à la logique du jeu : gestion des tentatives, détection de la victoire, calcul des points et validation des mots.

Une autre partie a été consacrée à l’affichage : grille, clavier, messages, couleurs et messages de fin de partie.

Nous avons aussi travaillé sur la récupération du mot secret avec le fichier JSON, ainsi que sur la création d’un dépôt fixe pour faciliter les tests.

Les choix importants ont été discutés au fur et à mesure, notamment pour l’organisation des classes, l’utilisation de l’interface *WordRepository* et la séparation entre logique du jeu et affichage.

13. Conclusion

Pour conclure, cette SAÉ nous a permis de réaliser une version fonctionnelle du jeu FindMyWord en Java. Le programme respecte les règles principales du sujet : validation des mots, analyse des lettres, gestion des tentatives, victoire, défaite et affichage de la grille.

Cette SAÉ nous a surtout permis d’avoir une meilleure vision de l’intérêt de la programmation orientée objet, car on a pu voir concrètement comment des classes séparées, avec des rôles précis, rendent un projet plus clair, plus organisé et plus facile à faire évoluer.